SNAP ANYTHING. FIGHT EVERYTHING.

One photo of any object becomes a **rigged, animated 3D fighter** with **Aholo Lux3D**, then brawls in an AI-generated **SpatialGen** world or a 3D Gaussian-splat scan of **your own room** (Aholo Reality, rendered with the Aholo Viewer). **RenderCloud** shoots each fighter's cinematic KO poster.




MUG SHOT


SOLE SURVIVOR


QUACK ATTACK


TOAST MASTER


PRICKLY PETE

POWERED BY AHOLO · LUX3D · SPATIALGEN · REALITY · AHOLO VIEWER · RENDERCLOUD · ASSET (OUS) · OFFICIAL TYPESCRIPT SDKS

TEAM

Gold Tek (solo)
Eromonsele Michael
GitHub: AshThunder

TRACK

Gaming

REPORT DATE

13 November 2026

LINKS

Live demo: anything-fighter.vercel.app
Gallery: anything-fighter.vercel.app/gallery
Source: github.com/AshThunder/anything-fighter

EXECUTIVE SUMMARY

A FIGHTING GAME WHOSE ROSTER IS THE WHOLE WORLD

Anything Fighter is a browser fighting game where the player supplies the characters. Point a phone at a mug, a sneaker or a rubber duck, press **Forge**, and a chain of Aholo services turns that single photo into a full-body fighter concept, a textured PBR mesh, a humanoid skeleton and a set of retargeted fight moves. A few minutes later the new fighter is in the roster, ready to fight the CPU, a friend on the same keyboard, or any other fighter in the gallery, inside an Aholo-generated 3D world.

Aholo is not an add-on here. It is the content pipeline: every fighter, every 3D arena, the room-scan stage and every KO poster comes from Manycore's tools, called through the official `@manycore/aholo-sdk-*` TypeScript packages. Everything else is built around them so the player only sees *photo in, fighter out*.

1 PHOTO
is all a player provides to forge a fighter

6
Aholo services in the live product (Asset/OUS, Lux3D, SpatialGen, Reality, Viewer, RenderCloud)

10
Lux3D-retargeted move clips per rigged fighter (idle, walk, jab, cross, hook, block, 2× hit, KO, victory)

THE PROBLEM

Fighting games live or die by their roster, and every character normally costs weeks of concept art, modelling, rigging and animation. Players can pick a costume, but never bring *their own* character, and certainly not the coffee mug on their desk.

Generative 3D has made single assets cheap, but a playable character needs much more than a mesh: it needs a humanoid body, a skeleton, animation that fits that skeleton, and a stage to fight in. Stitching that together by hand is still a studio job.

THE IDEA

Use Aholo's 3D AI stack as a **character and stage factory** that runs while the player waits:

- **Lux3D** re-imagines any object as a humanoid fighter, builds the mesh, rigs it and retargets preset fight moves.
- **SpatialGen** generates themed 3DGS arenas from text; **Reality** rebuilds the player's room from a phone video.
- **Aholo Viewer** renders those splats live behind the fight; **RenderCloud** path-traces a KO poster.

The result is a game where the roster grows with every photo, and every player has a fighter nobody else has.

WHAT A JUDGE CAN DO IN TWO MINUTES

#	ACTION	WHERE
1	Pick two starter fighters (each forged by Lux3D from one photo) and a SpatialGen or Reality arena, then press **FIGHT!**	Home → Select fighter → Select stage
2	Play combos, the special and the super; try 2-player on one keyboard or the touch pad on a phone	`/fight`
3	See every fighter's before → after: source photo, Lux3D concept, RenderCloud poster, live 3D turntable	`/gallery`
4	Forge a fighter from your own photo and spar with a practice partner while it forges	Home → Forge a fighter

PLAYER FLOW

SNAP → FORGE → PICK → FIGHT → SHARE

1
SNAP
Take or upload a photo of any object and name it. Photo sharing is **off** by default.

ASSET / OUS

2
FORGE
Each Aholo stage lights up as it finishes; spar with a partner while you wait.

LUX3D · R.CLOUD

3
PICK
YOU / CPU (or P1 / P2) slots, 3D versus preview, CPU level, stage.

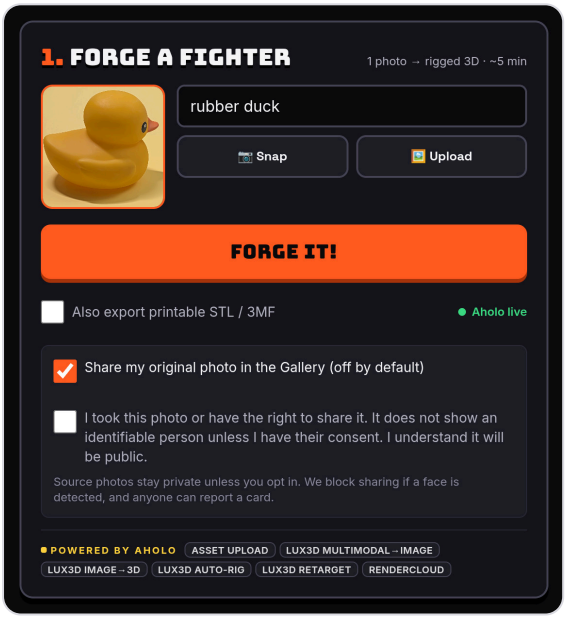
SPATIALGEN · REALITY

4
FIGHT
Rigged fighters move with their Lux3D clips over an Aholo Viewer 3DGS backdrop.

AHOLO VIEWER

5
SHARE
Download a KO card and a clip of the fight; browse every fighter in the gallery.

RENDERCLOUD



Forge panel. Snap or upload, one big FORGE button, optional printable export, opt-in photo sharing with a consent attestation, and the "Powered by Aholo" strip listing the services that will build this fighter.



Fighter select. Slot-based picking for you and the CPU, a live 3D versus preview of the two Lux3D fighters, and the Aholo services that built the selected fighter.

THE FORGE, STAGE BY STAGE

upload	concept	mesh	rig	retarget	poster	export	done
OUS	multimodal→image	img-to-3D	humanoid auto-rig	10 moves	RenderCloud	STL/3MF/USDZ, opt.	fighter.json

Stages as named in `lib/aholo.ts`. Poster and print export are non-fatal: a failure, or a poster taking over 8 minutes, moves the job on.

- **Concept.** Lux3D multimodal-to-image turns the object photo into a full-body humanoid fighting-game character in a T-pose, so that a mug can actually take a skeleton.
- **Mesh.** Lux3D image-to-3D, `G1-Turbo`, PBR textures, GLB output, requested as a 1.8 m character (`customSize: 1800`).
- **Rig + retarget.** Humanoid auto-rig returns a Mixamo-named 52-joint skeleton; one retarget task applies 10 official presets. Each clip is stripped to skeleton + animation for the browser.
- **Rescue path.** If Lux3D answers `RIG_ELIGIBILITY_FAILED`, the app re-rigs once on a normalised 1.8 m copy of the same mesh. If it still refuses, the fighter ships **freestyle** (unrigged Lux3D mesh, procedural moves) instead of dead-ending.

AHOLO AT THE CORE

EVERY AHOLO TOOL, AND WHAT IT DOES IN THE GAME

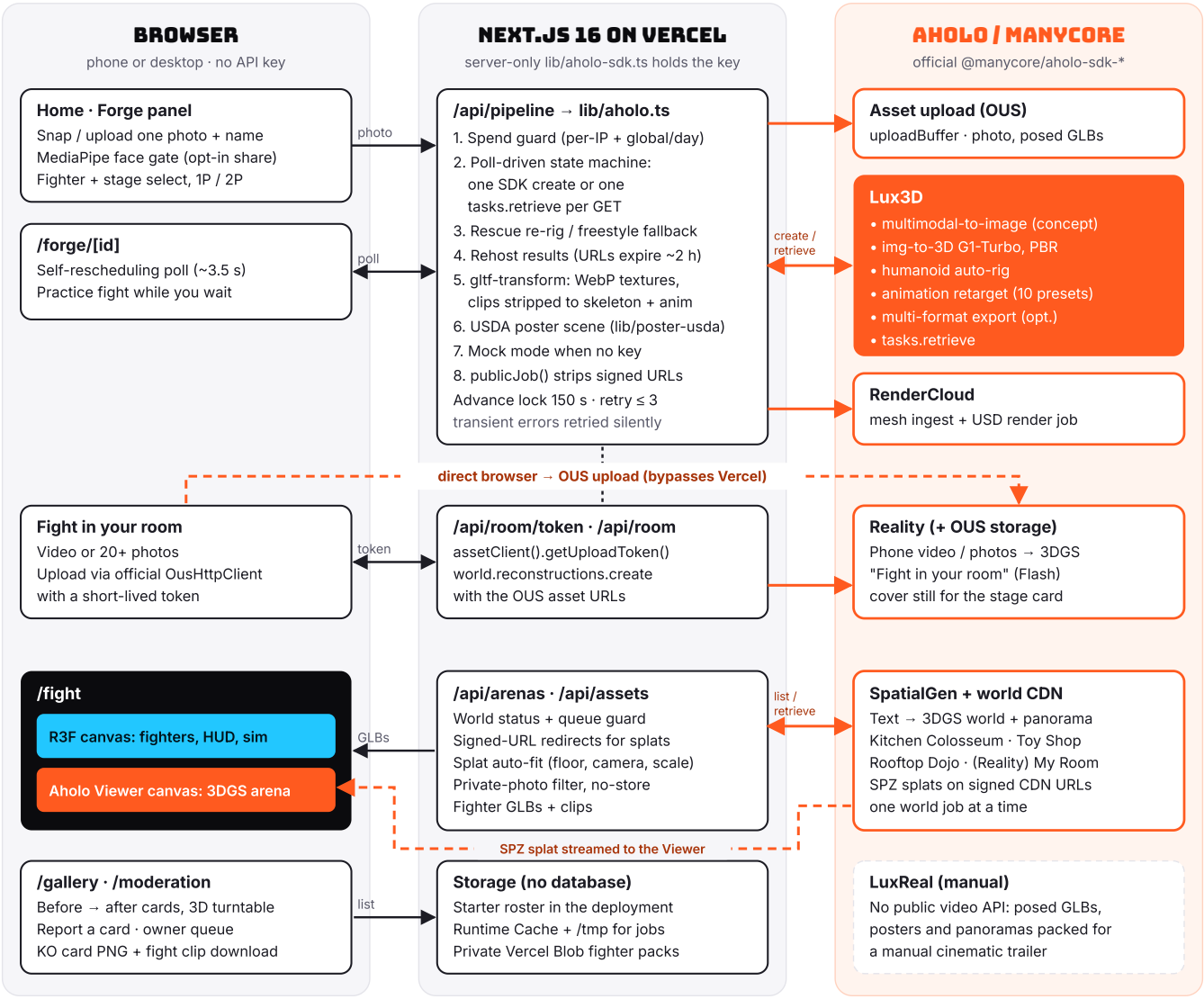
All Aholo calls in the deployed app go through Manycore's official TypeScript SDKs, created once in the server-only module `lib/aholo-sdk.ts` (global region `com`, `https://api.aholo.ai`). RenderCloud has no SDK, so its OpenAPI routes are sent through the official `@manycore/aholo-sdk-core` gateway client.

AHOLO TOOL	SDK CALL(S)	ROLE IN ANYTHING FIGHTER	CREDITS*
Asset upload (OUS) aholo-sdk-asset	`uploadBuffer`, `getUploadToken`; browser transfer via `OusHttpClient` (sdk-core)	Uploads the player's photo and the posed GLBs from the server; room-scan videos go straight from the browser to OUS with a short-lived token	—
Lux3D multimodal-to-image	`multimodalToImage.create`	Object photo → full-body humanoid fighter concept (T-pose) so any object can be rigged	~12
Lux3D image-to-3D	`imgTo3d.create` (G1-Turbo, PBR, GLB)	Concept → textured mesh at character scale (1.8 m)	~1
Lux3D humanoid auto-rig	`humanoidAutoRig.create`	Mixamo-named 52-joint skeleton on the generated mesh	~15
Lux3D animation retarget	`humanoidAnimationRetarget.create`	10 preset moves per fighter: `Idle_Loop`, `Walk_Loop`, `Punch_Jab`, `Punch_Cross`, `Melee_Hook` (kick), `Sword_Block`, `Hit_Chest`, `Hit_Knockback`, `Death01`, `Dance_Loop`	~3 / clip
Lux3D multi-format export	`multiFormatExport.create`	Optional printable STL / 3MF / USDZ of your fighter (checkbox, off by default)	—
Lux3D tasks	`tasks.retrieve`	One status check per client poll drives the forge state machine	—
SpatialGen aholo-sdk-world	`generations.create`, `retrieve`, `list`	Text → 3D Gaussian-splat arenas + panoramas: Kitchen Colosseum, Toy Shop at Midnight, Rooftop Dojo at Sunset	10 / world
Reality aholo-sdk-world	`reconstructions.create`, `retrieve`	"Fight in your room": a 10–20 s phone video (or 20+ photos) → playable 3DGS arena (Flash quality)	20 / scan
Aholo Viewer	`@manycore/aholo-viewer`	Renders the SPZ/SOG splat arenas under the transparent React Three Fiber fight canvas, camera synced every frame	—
RenderCloud OpenAPI via sdk-core	mesh-upload-process task, `/rendercloud/v1/jobs`	Path-traced KO / victory poster: mid-jab pose baked from the punch clip, USD scene with studio backdrop and lights	non-fatal stage
LuxReal	web app (no public API)	Manual cinematic trailer pack assembled by `scripts/luxreal-pack.mjs`; not part of the live app	—

* Approximate credit costs as recorded in the project's handoff notes during development; the Aholo dashboard is authoritative. A full live forge is on the order of **~35–60 credits**. The hackathon grant was 3,600 credits, which is why live forges are capped by a spend guard (page 10).

SYSTEM DESIGN

BROWSER · NEXT.JS ON VERCEL · AHOLO CLOUD



Solid arrows: HTTP calls between the browser and the app's routes; orange arrows: calls into Aholo through the official SDKs (RenderCloud via the sdk-core gateway client). The API key exists only on the server; the browser only ever receives a short-lived OUS upload token.

HOW THE INTEGRATION FOLLOWS MANYCORE'S GUIDANCE

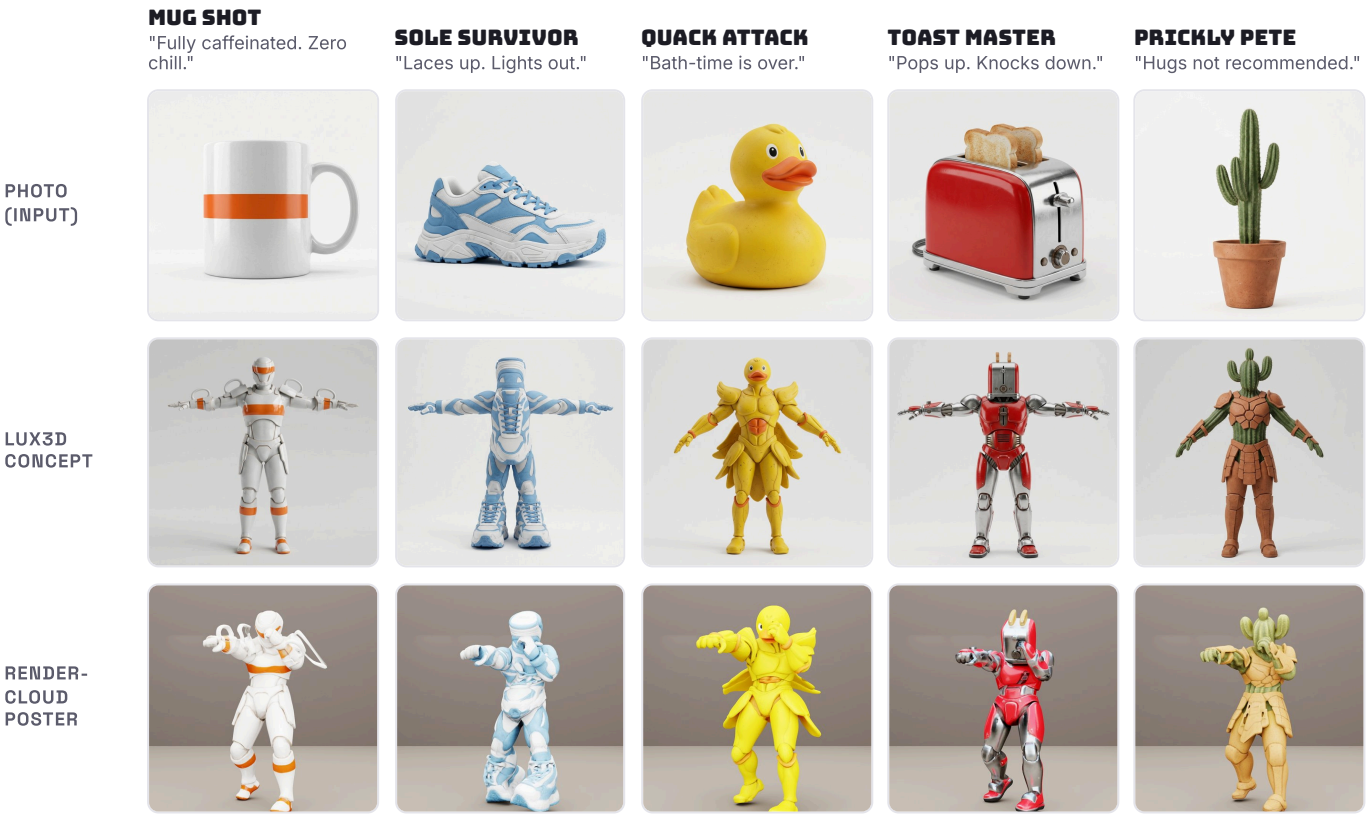
- **SDKs, not hand-written HTTP** for Asset, World and Lux3D, as Manycore's `llms.txt` recommends. SDK errors (`RateLimitError`, `BusinessError`…) are mapped to app errors; 429 / 5xx / network are retried on the next poll.
- **Async create → poll**, exactly as the World and Lux3D APIs are designed, but one step per request (page 11).
- **Visible in the product:** a "Powered by Aholo" strip on the forge panel, the live forge page (lights up per stage), the select screen, the VS splash and every gallery card lists the services that actually built that fighter.
- **No key, still demoable:** without an API key every pipeline runs in a clearly labelled mock mode.

Stack: Next.js 16 (App Router, TypeScript, Tailwind 4) · React 19 · React Three Fiber + drei + three.js · Zustand · `@manycore/aholo-sdk-asset` / `-world` / `-lux3d` / `-core` · `@manycore/aholo-viewer` · gltf-transform + meshoptimizer · sharp · MediaPipe Tasks Vision (face gate) · Vercel Functions, Runtime Cache and Blob · WebAudio-synthesised sound.

ASSET ITERATIONS

FROM KITCHEN OBJECT TO CONTENDER

The five starter fighters were forged with the same Lux3D chain the live app uses (source photos were themselves AI-generated for this project; no third-party photos). Each row is one photo, its Lux3D concept and its RenderCloud poster, the same before → after shown on every gallery card.



What we learned from Lux3D

Asking multimodal-to-image for a **full-body humanoid in a T-pose** is what makes rigging work on non-humanoid objects. Requesting a **1.8 m character** (instead of letting size be predicted from the object, which sized a sneaker like a 30 cm figurine) was the fix for rig eligibility failures.

Rig & retarget findings

The rig is a Mixamo-named 52-joint skeleton. Each retargeted clip GLB repeats the full textured mesh (~22 MB), so the app strips it to skeleton + clip (≈0.05–0.1 MB) and converts fighter textures to WebP (21.8 MB → 3.4 MB in the measured run). The 85 official presets include no kick, so `Melee_Hook` stands in.

How the KO poster is made

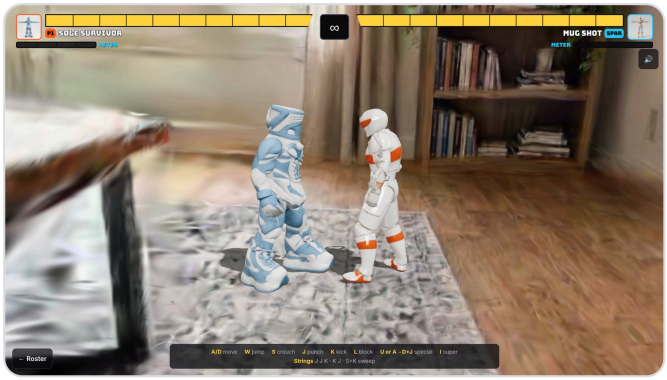
The app bakes a mid-jab pose from the fighter's own punch clip into a static mesh, uploads it through OUS, runs RenderCloud mesh ingest, builds a USD scene (studio backdrop + lights, `lib/poster-usda.ts`) and submits an offline path-traced render job. The poster is shown on the VS splash, in the gallery and on the downloadable KO card.

3D WORLDS

FIGHT IN AN AI WORLD, OR IN YOUR OWN ROOM



Rooftop Dojo at Sunset (SpatialGen 3DGS world) · Mug Shot vs Quack Attack.



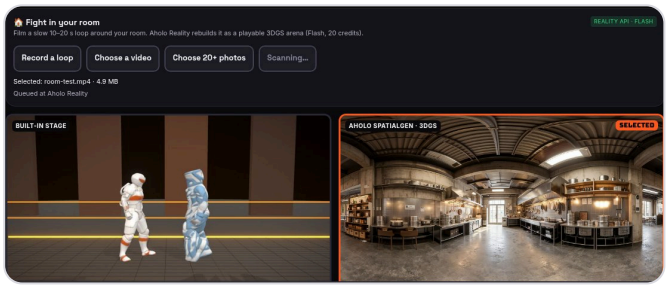
My Room (scan) (Aholo Reality reconstruction) · Sole Survivor vs Mug Shot.

SPATIALGEN ARENAS

Three themed worlds were generated from our own text prompts with `world.generations.create`: **Kitchen Colosseum**, **Toy Shop at Midnight** and **Rooftop Dojo at Sunset**. Each produces a 3DGS splat and a panorama; the panorama becomes the stage-select cover and the fallback backdrop. Finished worlds appear as playable stage cards automatically.

AHOLO REALITY: "FIGHT IN YOUR ROOM"

The stage select has a builder card: record a slow 10–20 s loop (or choose a video, or 20+ photos), and Aholo Reality rebuilds the room as a playable splat arena (Flash quality, ~20 credits). Files upload **directly from the browser to Aholo OUS**; the server only mints the upload token and calls `world.reconstructions.create`. Reality worlds use the official `WorldDetail.cover` as their card image.



Stage select with the always-visible Reality room builder above the built-in Neon Ring and SpatialGen cards.

AHOLO VIEWER UNDER REACT THREE FIBER

The Aholo Viewer is its own WebGL engine and cannot live inside an R3F canvas, so the game stacks two canvases: the Viewer renders the splat underneath, and a transparent R3F canvas draws the fighters on top, with the Viewer camera synced from the fight camera every frame.

> **Making splats fight-ready.** `lib/splat-fit.ts` decodes each SPZ, finds the floor band and the largest floor rectangle, aims the camera down the room's long axis and scales the world so fighters stand on the floor. New worlds are auto-fitted server-side. If a splat cannot load on a device, the fight swaps to the world's panorama, then to the Neon Ring, and keeps going.

ARENA	BUILT WITH	KIND
Kitchen Colosseum	Aholo SpatialGen (text prompt)	3DGS splat + panorama
Toy Shop at Midnight	Aholo SpatialGen (text prompt)	3DGS splat + panorama
Rooftop Dojo at Sunset	Aholo SpatialGen (text prompt)	3DGS splat + panorama
My Room (scan)	Aholo Reality (phone video of a room)	3DGS splat
Aholo Sample Splat · Neon Ring	Official Aholo Viewer sample · built-in stage	always-available fallbacks

FUN & ENGAGEMENT

A REAL FIGHTING GAME, NOT A MODEL VIEWER



Kitchen Colosseum (SpatialGen world) · Mug Shot vs Quack Attack, 1P vs CPU. Before each fight a VS splash slams in both RenderCloud posters and tags each fighter with how it was built (e.g. "Aholo Lux3D starter · retargeted clips").

MOVE SET

INPUT	MOVE
P P K	jab → cross → hook string, knockdown
K P	kick → rush, cancels on hit
↓ + P / ↓ + K	low punch / sweep (must be crouch-blocked)
P or K in the air	overhead jump attack (block standing)
Block just before a hit	parry: attacker stunned, +meter
Special (U, or back→forward+P)	armoured dash-strike, knockdown
Super (I, meter full)	slow-mo, camera zoom, 3-hit armoured combo

Plus jump and crouch: highs whiff over a crouch; any clean hit on an airborne fighter is an anti-air knockdown. Damage scales down over long combos; counter-hits deal +20 %.

MODES & FEEL

- **1P vs CPU** with Easy / Normal / Hard. The CPU drives the same inputs as a player: it extends strings, punishes whiffs, parries (Hard), jumps in, sweeps and anti-airs.
- **2 players on one keyboard**, and **practice** (infinite time, the partner never attacks), shown while your fighter forges.
- **Any fighter vs any fighter:** starters, your forged fighter, or one from the gallery ("Fight as" / "Fight vs").
- **Arcade presentation:** screenpack-style life bars with a lagging damage layer, super meter ring, round timer, combo and hit callouts, hit sparks, screen shake, haptics on phones, K.O. / TIME! stamp, victory panel.
- **Freestyle fighters still fight:** if Lux3D can't rig a mesh, procedural motion (and a 3-bone arm rig for T-posed meshes) performs every move.
- **After the fight:** rematch, a 1600×900 KO card with the RenderCloud poster, and a recorded clip (MP4 or WebM) to share manually.

The rules are covered by `npm run test:sim`: 18 deterministic checks (strings, scaling, jump arc, crouch, lows, overheads, anti-air, special, super).

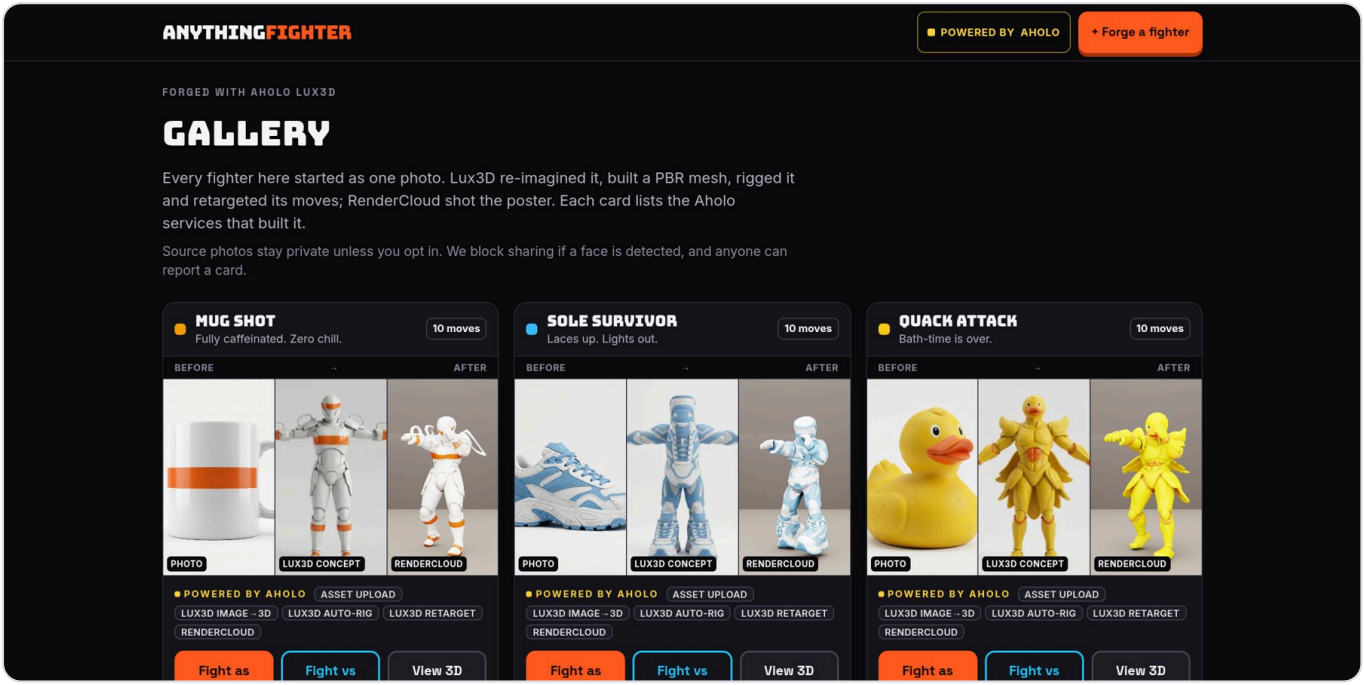
CONTROLS

	MOVE	JUMP / CROUCH	PUNCH	KICK	BLOCK	SPECIAL	SUPER
Solo	A / D or ← →	W, ↑, Space / S, ↓	J	K	L, Shift, C	U	I
2P · P1	A / D	W / S	F	G	H	R	T
2P · P2	← / →	↑ / ↓	K / Num1	L / Num2	M / Num0	O / Num3	P / Num6
Touch	◀ ▶	▲ Jump / ▼ Crouch	Punch	Kick	Block	Special	Super (ring)

GALLERY & SHARING

EVERY FIGHTER SHOWS ITS TRANSFORMATION

The gallery (`/gallery`) is where the Aholo pipeline is most visible. Every fighter, starter or player-forged, gets a card that tells its whole story, so a visitor understands in one glance what Lux3D and RenderCloud did with a single photo.



Gallery on desktop: before → after strip (photo | Lux3D concept | RenderCloud poster), the "Powered by Aholo" services that built each fighter, and Fight as / Fight vs / View 3D.

Before → after

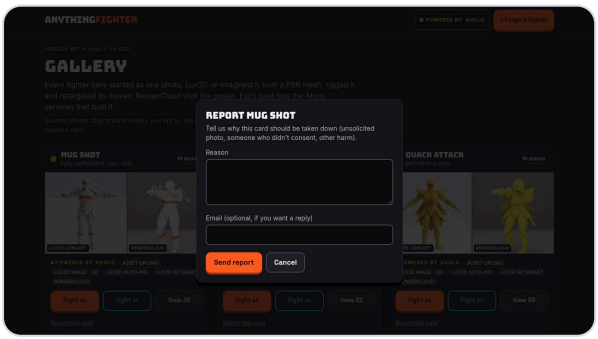
Photo, Lux3D concept and RenderCloud poster side by side. A player's photo appears only if they opted in and passed the face gate; otherwise the card says it is kept private and shows concept and poster.

Play from the gallery

Fight as or **Fight vs** any card, including a friend's freshly forged fighter, or open a live 3D turntable of the rigged Lux3D model.

Share outside the game

After a fight: a 1600×900 KO card PNG built around the RenderCloud poster and a recorded clip of the fight. Nothing is posted automatically; the player shares it themselves.



Report this card: every gallery card can be flagged; reports go to an owner-only moderation queue.

ALWAYS VISIBLE: "POWERED BY AHOLO"

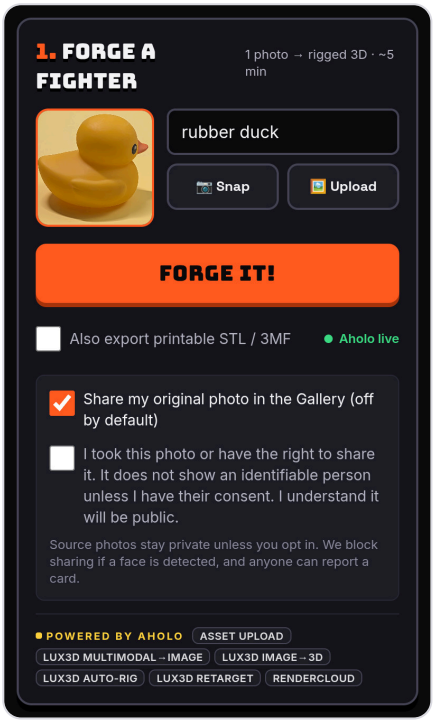
The same services strip appears on the forge panel (planned services), on the live forge page (each service lights up as its stage finishes), on the fighter select screen, on the VS splash and on every gallery card. Fighters are tagged by how they were built, for example "Forged with Lux3D · rig + N Aholo clips" or "Forged with Lux3D · freestyle moves", so judges and players can see exactly which Manycore tools produced what.

RESPONSIBLE DESIGN

SNAP ANYTHING, BUT PROTECT THE PEOPLE IN IT

"Snap anything" invites photos of people, so the design assumes that will happen. Nothing a player uploads becomes public unless they explicitly ask, attest and pass the face gate, and anyone can flag what does.

SAFEGUARD	HOW IT WORKS
Private by default	A player's source photo is never served or listed unless they opt in; thumbnails come from the Lux3D concept or the RenderCloud poster.
Opt-in + consent	Sharing needs a checkbox *and* an attestation that the uploader has the right to share and the photo shows no identifiable person without their consent. The choice is recorded at forge time; older forges are never published retroactively.
Face gate	Aholo's published APIs have no face-detection or moderation endpoint, so the browser runs MediaPipe face detection. A face, or any model error, keeps the photo private (fail closed); the forge itself still runs.
Report + moderation	Every card has a Report dialog; reports land in an owner-only queue at `/moderation` that can hide or unpublish a card.
One-time delete link	When a forge finishes, its creator gets a one-time "Remove from gallery" control backed by a signed token.
Instant takedown	Shared photos are served with `private, no-store`, so a removal takes effect immediately rather than lingering in caches.
No leaked URLs	Job responses strip signed Aholo URLs (including the uploaded photo); the durable Blob store is private.
Spend guard	Live forges are capped per IP and globally per Africa/Lagos day; past the cap the forge continues as a clearly labelled demo forge (no credits) instead of failing.
Secrets	The Aholo API key lives only in server-only modules; the browser gets at most a short-lived OUS upload token. Without a key the whole app runs in labelled mock mode.



Forge form on a phone (live site): sharing is off by default and needs the consent attestation.

Why the face check runs in the browser. We looked for a moderation or face-detection capability in Aholo's published OpenAPI (Asset, World, Lux3D, RenderCloud, Spatial Understanding) and found none, so the check runs on the player's device before anything can be shared. It only gates *public sharing*; the photo still goes to Lux3D to build the fighter, and stays private.

COMPLETENESS

PROBLEMS WE HIT, AND HOW WE SOLVED THEM

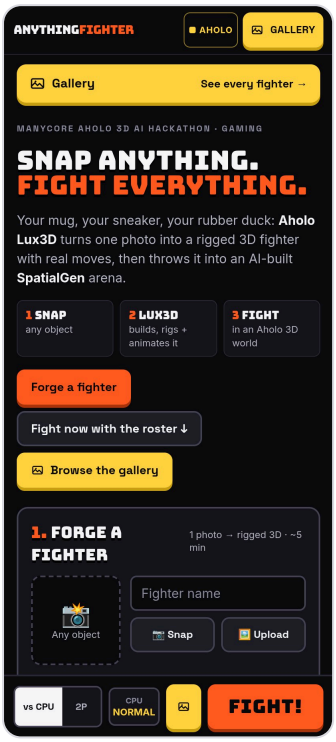
CHALLENGE	SOLUTION
Long async Aholo jobs vs short serverless requests	**Poll-driven state machine.** Each `GET /api/pipeline/[id]` performs at most one transition: one SDK `create` or one `tasks.retrieve`. The SDKs' blocking `waitFor` helpers are deliberately not used inside requests, so no function waits on a multi-minute job. An advance lock (150 s) stops a hung function from wedging a job.
Forge froze on mobile ("Failed to fetch")	The forge page's poll is a **self-rescheduling loop** with exponential backoff, a 75 s request abort, and immediate resume on tab focus, visibility change or network return. Transient errors (429, 5xx, network, timeouts) retry silently; players see plain-language messages with a Retry.
Room scans vs Vercel's ~4.5 MB request body limit	A real scan video is ~18 MB, so posting it through a serverless function could never work in production. Now the server mints a **short-lived OUS token** with the official asset SDK, the browser uploads straight to **Aholo OUS** with the official `OusHttpClient` (single / multipart, MD5, status poll), and the server calls `world.reconstructions.create` with the asset URLs. The API key never leaves the server.
Lux3D refused some rigs (`RIG_ELIGIBILITY_FAILED`)	Request 1.8 m characters; on refusal, re-rig once on a normalised copy of the same mesh (no new mesh credits); if it still fails, ship the fighter **freestyle** with procedural animation rather than dead-ending the forge.
Heavy rig / clip outputs	Each ~22 MB retarget GLB repeats the mesh; the app downloads one clip per poll and strips it to skeleton + clip with gltf-transform; fighter textures go to WebP.
Aholo result URLs expire (~2 h)	Everything needed later is rehosted. Finished world splats are served through a redirect to a freshly signed Aholo URL (no multi-MB rehosting).
Ephemeral serverless storage	Starters ship in the deployment; jobs use Runtime Cache (2 MB items chunked) + `/tmp`; finished live fighters are saved as one private **Vercel Blob** pack. Missing models are hidden from the roster, and a `SafeModel` boundary swaps a failing model or clip for procedural moves.
Aholo Viewer + Next.js bundling	Bundling the Viewer broke its Blob-URL workers; it is vendored to `public/vendor` and loaded with a non-bundled `import()`. Splat URLs are made absolute for the worker; each URL is retried with a same-origin proxy as a fallback.
One world job per account, no cancel API	`/api/arenas` reports the account queue, refuses new world jobs while it is busy, and flags jobs stuck at 0 % for 90+ minutes. Fights always have Neon Ring and the Aholo Viewer sample splat available.
Limited hackathon credits	Per-IP and global daily forge limits; past the cap the forge continues in labelled mock mode, so judges never hit a dead end and the budget is protected.

HOW WE TESTED IT

- `npm run test:sim`: 18 deterministic gameplay checks run offline (no credits).
- `npm run test:durable`: the Blob pack store against a mocked Blob API, including cache headers and the health probe.
- Lint and production builds; production checks of `/api/health`, photo privacy (private photos return 404) and cache headers.
- Scripted browser drivers playing full fights with keyboard and real touch-pointer events, including Android emulation (412×915, touch, 4× CPU throttle) against the Hard CPU.
- Live end-to-end forges through the official SDKs on production, kept few on purpose to protect the credit budget.
- Mock mode lets the whole UI be exercised without spending credits.

MOBILE SUPPORT

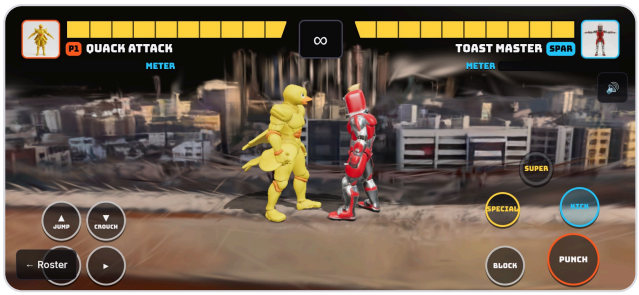
BUILT PHONE-FIRST



Home on a 412 px phone.



Toy Shop fight with the touch pad.



Landscape: Quack Attack vs Toast Master on Rooftop Dojo.

- **Snap** uses the phone camera; room scans have separate Record and Upload buttons so Android can pick a video from the gallery.
- **Touch pad:** ▲ Jump, ▼ Crouch, ◀ ▶, a large Punch, Kick, Block, Special and a Super button with the meter ring; all targets ≥ 56 px, fits 360 px screens, safe-area padded.
- Camera field of view adapts to aspect ratio; short move hints for the first seconds on touch; vibration on hits.
- Splat arenas retry and fall back to panorama or Neon Ring on devices that can't load them.

FUTURE WORK

- Try Lux3D `imageToFourView` → multi-view `imgTo3d` (G1) for higher-fidelity meshes (wired in the SDK wrapper, not yet used).
- LuxReal cinematic KO replays and trailers once a video API or deeper integration is available.
- Best-of-3 rounds with win icons, gamepad support, juggles built on the anti-air knockdown.
- Migrate the offline CLI scripts to the official SDKs; grey out stage cards in demo forges.
- Online play between two forged fighters.

CREDITS & LICENCES

Fighters generated by Aholo Lux3D from AI-generated source photos; arenas generated by Aholo SpatialGen from our prompts; the room-scan sample video was AI-generated for this project. Placeholder fallback: RobotExpressive by Tomás Laulhé (CC0). Sound and music synthesised at runtime with WebAudio (original). Fonts: Bungee, Space Grotesk, Inter (SIL OFL 1.1).

LINKS

Live demo	https://anything-fighter.vercel.app
Gallery	https://anything-fighter.vercel.app/gallery
Source code (public)	https://github.com/AshThunder/anything-fighter
Aholo / Manycore docs used	https://labs.aholo.ai/api-docs/en/ · TypeScript SDK: https://labs.aholo.ai/api-docs/en/sdk/typescript
Team	Gold Tek (solo) · Eromonsele Michael · GitHub AshThunder · Track: Gaming